**ТОО «Колледж Хекслет»**

# ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

**на тему:**

«Разработка веб-приложения «GoalTracker»
для отслеживания личных целей»

**по модулю:** ПМ2 «Прикладное программирование»

**Специальность:** 06130100 – Программное обеспечение (по видам)

**Квалификация:** 4S06130105 – Техник информационных систем

**Выполнил(-а): студент 2 курса**
13ТИС группы
Степанов Тим Александрович

**Допущен(-а) к защите «_____» _____ 20____ г.**

**Преподаватель: Бенли Р.А.**

_____
(подпись)

**Защитил(-а) с оценкой:**

(буква) (циф.экв.) (балл)

**Алматы 2026**

# СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ 5
1.1 Описание инструментов и среды разработки 5
1.2 Постановка задачи (техническое задание) 7
1.3 Проектирование структуры данных 10
1.4 Проектирование интерфейса приложения (UI/UX) 12
1.5 Описание внутренней структуры приложения (архитектура) 14
2. ПРАКТИЧЕСКАЯ ЧАСТЬ 16
2.1 Создание проекта и структуры файлов 16
2.2 Реализация слоя хранения данных (storage.js) 17
2.3 Реализация вспомогательных функций (utils.js) 18
2.4 Реализация основной логики приложения (main.js) 19
2.5 Реализация формы создания цели (form.js) 20
2.6 Реализация модуля статистики (stats.js) 21
2.7 Разработка стилей приложения (style.css) 22
2.8 Обработка ошибок и валидация данных 23
ЗАКЛЮЧЕНИЕ 24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 25
ПРИЛОЖЕНИЯ 26

# ВВЕДЕНИЕ

В современном мире эффективное управление личными целями является одним из ключевых факторов успешной учебной и профессиональной деятельности. Умение ставить чёткие цели, планировать их достижение и контролировать прогресс — навыки, необходимые каждому человеку, стремящемуся к саморазвитию.

Учебная практика по профессиональному модулю ПМ2 «Прикладное программирование» направлена на формирование практических компетенций в области разработки программных продуктов. В ходе практики студент самостоятельно разрабатывает полноценное программное приложение, демонстрируя полученные теоретические и практические знания.

Актуальность темы данного проекта обусловлена высоким спросом на инструменты личной продуктивности. Веб-приложения для постановки и отслеживания целей пользуются значительной популярностью: по данным аналитиков, рынок приложений для управления задачами ежегодно растёт на 12–15%. Разработка подобного приложения требует комплексного применения знаний в области клиентских веб-технологий: HTML5, CSS3 и JavaScript ES6+.

**Цель практики:**

Разработка веб-приложения «GoalTracker» — системы постановки и отслеживания личных целей с поддержкой категоризации, приоритизации, дедлайнов, отображения прогресса и статистики.

**Задачи практики:**

– изучить современные клиентские веб-технологии (HTML5, CSS3, JavaScript ES6+);

– спроектировать структуру данных для хранения целей в localStorage;

– разработать модульную архитектуру клиентского приложения;

– реализовать CRUD-операции (создание, чтение, обновление, удаление) для целей;

– создать адаптивный интерфейс с поддержкой фильтрации и поиска;

– реализовать модуль статистики и аналитики целей;

– обеспечить валидацию данных и обработку ошибок ввода.

**Ожидаемые результаты:**

Функциональное веб-приложение, работающее полностью в браузере без серверной части, сохраняющее данные между сессиями и предоставляющее пользователю полный набор инструментов для управления личными целями.

**Методы работы:**

– изучение официальной документации MDN Web Docs по HTML, CSS и JavaScript;

– применение принципов модульного программирования при организации кода;

– итеративная разработка с поэтапным тестированием каждого компонента;

– использование системы контроля версий Git для отслеживания изменений.

# 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

## 1.1 Описание инструментов и среды разработки

Для реализации проекта GoalTracker был выбран стек технологий, обеспечивающий разработку полноценного клиентского веб-приложения без необходимости серверной инфраструктуры.

### HTML5

HTML5 (HyperText Markup Language версии 5) — стандартный язык разметки для создания веб-страниц. В проекте используются следующие возможности HTML5:

– семантические теги <nav>, <main>, <section>, <header>;

– атрибуты форм: type="date", type="number", required, min, max;

– атрибуты данных data-* для хранения идентификаторов в DOM;

– localStorage API для клиентского хранения данных.

### CSS3

CSS3 (Cascading Style Sheets) — язык описания внешнего вида HTML-документа. В проекте CSS3 применяется для:

– реализации адаптивной сетки (CSS Grid Layout, Flexbox);

– анимации и переходов (transition, transform);

– медиа-запросов для адаптивного дизайна (breakpoint 768px);

– переменных CSS (--color-primary, --color-accent и др.).

### JavaScript ES6+

JavaScript — основной язык программирования приложения. Используются возможности стандарта ES6 и выше:

– модульная организация через отдельные .js файлы (без bundler);

– стрелочные функции, деструктуризация, шаблонные строки;

– методы массивов: filter(), map(), reduce(), find();

– работа с localStorage через JSON.stringify / JSON.parse;

– генерация уникальных идентификаторов через Date.now() и Math.random().

**Среда разработки и инструменты**

| Инструмент | Версия / описание | Назначение |
|---|---|---|
| Visual Studio Code | 1.87+ | Основная IDE |
| Node.js / npm | v25.6.1 / v11.9.1 | Менеджер пакетов |
| Git | 2.53 | Контроль версий |
| GitHub | — | Хостинг репозитория |
| Google Chrome | последняя | Тестирование |
| Google Fonts | Inter | Типографика |
| markdown-pdf | 11.0.0 | Генерация документации |

*Таблица 1 – Инструменты разработки*

## 1.2 Постановка задачи (техническое задание)

### 1.2.1 Общее описание системы

GoalTracker — клиентское веб-приложение для постановки и отслеживания личных целей. Приложение работает полностью в браузере пользователя без серверной части. Данные хранятся в localStorage браузера между сессиями. Интерфейс реализован на русском языке и адаптирован для устройств с разными размерами экрана.

### 1.2.2 Функциональные требования

– ФТ-1: Просмотр списка всех целей на главной странице с отображением ключевых атрибутов.

– ФТ-2: Создание новой цели через форму с полями: название, категория, дедлайн, приоритет, прогресс, описание.

– ФТ-3: Удаление цели из системы с подтверждением действия пользователем.

– ФТ-4: Изменение статуса цели (активна / выполнена) одним нажатием.

– ФТ-5: Поиск целей по названию в режиме реального времени.

– ФТ-6: Фильтрация целей по категории (Учёба, Спорт, Здоровье, Финансы, Саморазвитие).

– ФТ-7: Фильтрация целей по статусу (все / активные / выполненные).

– ФТ-8: Просмотр статистики: общее количество, выполненные, активные, процент выполнения, популярная категория.

– ФТ-9: Автоматическая инициализация тестовыми данными при первом запуске.

– ФТ-10: Навигация между страницами через меню.

### 1.2.3 Нефункциональные требования

– НФТ-1: Время загрузки страницы — не более 2 секунд.

– НФТ-2: Отклик на действие пользователя — не более 100 миллисекунд.

– НФТ-3: Поддержка браузеров: Chrome 90+, Firefox 88+, Edge 90+.

– НФТ-4: Адаптивный дизайн для экранов от 320px до 1920px.

– НФТ-5: Работа без подключения к интернету (offline-режим, кроме шрифтов).

– НФТ-6: Код организован в модули с чётким разделением ответственности.

– НФТ-7: Интерфейс на русском языке.

### 1.2.4 Описание входных и выходных данных

Входные данные: информация о цели, вводимая пользователем через форму. Все поля, кроме описания, являются обязательными. Выходные данные: отображение списка целей, карточек целей и статистических показателей.

| Поле | Тип | Обязательное | Ограничения |
|---|---|---|---|
| Название (title) | text | Да | Непустая строка |
| Категория (category) | select | Да | Одно из 5 значений |
| Дедлайн (deadline) | date | Да | Формат YYYY-MM-DD |
| Приоритет (priority) | select | Да | Low / Medium / High |
| Прогресс (progress) | number | Да | 0–100 |
| Описание (description) | textarea | Нет | Любой текст |

*Таблица 2 – Поля формы создания цели*

## 1.3 Проектирование структуры данных

### 1.3.1 Выбор способа хранения данных

Поскольку приложение GoalTracker является клиентским и не предполагает серверной части, для хранения данных используется Web Storage API — конкретно localStorage. Это позволяет данным сохраняться между сессиями браузера без необходимости подключения к серверу или базе данных.

Обоснование выбора localStorage:

– данные сохраняются между закрытием и открытием браузера (в отличие от sessionStorage);

– объём хранения — до 5 МБ на домен, что достаточно для сотен целей;

– встроенный API браузера — не требует дополнительных библиотек;

– синхронный доступ — данные доступны мгновенно без асинхронных запросов.

### 1.3.2 Описание предметной области

Основная сущность системы — **Цель (Goal)**. Цель описывает намерение пользователя достичь определённого результата в заданные сроки. Каждая цель характеризуется названием, категорией, сроком выполнения, приоритетом, текущим прогрессом и статусом.

### 1.3.3 Схема объекта Goal (JSON-структура)

| Поле | Тип данных | Описание | Пример значения |
|---|---|---|---|
| id | string | Уникальный идентификатор | 1712345678abc |
| title | string | Название цели | Выучить React |
| category | string | Категория цели | Study |
| deadline | string | Срок выполнения | 2026-06-01 |

| | | | |
|---|---|---|---|
| priority | string | Приоритет | High |
| progress | number | Прогресс, % | 45 |
| description | string | Описание (опц.) | Изучить хуки |
| completed | boolean | Статус выполнения | false |
| createdAt | string | Дата создания | 2026-01-15 |

*Таблица 3 – Схема объекта Goal*

### 1.3.4 Категории целей

| Значение (англ.) | Отображение | Описание |
|---|---|---|
| Study | Учёба | Образовательные цели |
| Sport | Спорт | Физическое развитие |
| Health | Здоровье | Здоровый образ жизни |
| Finance | Финансы | Финансовые цели |
| Self-development | Саморазвитие | Личностный рост |

*Таблица 4 – Категории целей*

## 1.4 Проектирование интерфейса приложения (UI/UX)

### 1.4.1 Структура навигации

Приложение состоит из четырёх HTML-страниц, связанных общей навигационной панелью (nav). Пользователь может перейти на любую страницу в один клик. Активная страница визуально выделяется в меню.

| Страница | Файл | Назначение | Основные элементы |
|---|---|---|---|
| Главная | index.html | Список и управление целями | Поиск, фильтры, карточки целей |
| Добавить | form.html | Создание новой цели | Форма с валидацией |
| О проекте | about.html | Информация о приложении | Описание, автор, инструкция |
| Статистика | stats.html | Аналитика | Карточки с метриками |

*Таблица 5 – Страницы приложения*

### 1.4.2 Описание элементов управления

На главной странице (index.html) размещены элементы фильтрации и поиска:

– Поле поиска (#search) — фильтрует цели по вхождению текста в название в реальном времени (событие input);

– Выпадающий список (#categoryFilter) — фильтрация по категории (5 значений + «Все категории»);

– Выпадающий список (#statusFilter) — фильтрация по статусу (Все / Активные / Выполненные);

– Кнопка «Добавить цель» (add-goal-btn) — переход на форму создания;

– Карточки целей (goal-card) — отображают все атрибуты, кнопки «Выполнена» и «Удалить».

### 1.4.3 Цветовая схема и типографика

| Элемент | Цвет (HEX) | Назначение |
|---|---|---|
| Навигация / шапка | #1f3c88 | Тёмно-синий: брендовый цвет |
| Кнопки / ссылки | #3b82f6 | Синий: основные действия |
| Успех / добавление | #22c55e | Зелёный: подтверждение |
| Удаление | #ef4444 | Красный: деструктивные действия |
| Нейтральные | #6b7280 | Серый: второстепенные элементы |
| Фон страницы | #f4f7fb | Светло-голубой: комфортный фон |
| Карточки | #ffffff | Белый: контент на фоне |

*Таблица 6 – Цветовая схема приложения*

### 1.4.4 Адаптивный дизайн

Приложение адаптировано под экраны различных размеров с помощью CSS-медиа-запроса на breakpoint 768px. При ширине экрана менее 768px:

– навигационные ссылки выстраиваются вертикально с wrap;

– элементы фильтрации занимают 100% ширины и располагаются вертикально;

– сетка карточек (CSS Grid) переключается в одноколоночный вид;

– кнопки формы располагаются вертикально.

## 1.5 Описание внутренней структуры приложения (архитектура)

### 1.5.1 Общая архитектура

Приложение GoalTracker построено по принципу **модульной архитектуры** с чётким разделением ответственности. Каждый JavaScript-модуль отвечает за конкретный аспект работы приложения. Взаимодействие между модулями происходит через глобально доступные функции и общий localStorage.

### 1.5.2 Описание модулей

| Модуль | Файл | Ответственность | Основные функции |
|---|---|---|---|
| Storage | storage.js | Уровень данных: чтение/запись localStorage | getGoals(), saveGoals(), initializeGoals() |
| Utils | utils.js | Вспомогательные утилиты | generateId(), formatDate(), calculateCompletedGoals(), getCategoryStats() |
| Main | main.js | Основная логика: отображение, удаление, переключение | renderGoals(), createGoalCard(), deleteGoal(), toggleGoalStatus(), filterGoals() |
| Form | form.js | Обработка формы создания цели | validateForm(), createGoalObject() |
| Stats | stats.js | Расчёт и отображение статистики | renderStats() |

*Таблица 7 – Модули приложения*

### 1.5.3 Схема взаимодействия компонентов

Ниже представлена схема потока данных в приложении:

```
[redacted] → [HTML-[redacted]] → form.js (validateForm)
    ↓
form.js → createGoalObject() → storage.js (saveGoals)
    ↓
localStorage ← JSON.stringify(goals)
    ↓
main.js (renderGoals) ← storage.js (getGoals) ← localStorage
    ↓
[DOM: goalsContainer] → createGoalCard() → [redacted] [redacted]
    ↓
[redacted] ([redacted]/[redacted]) → deleteGoal/toggleGoalStatus → saveGoals
    ↓
stats.js → renderStats() ← calculateCompletedGoals(), getCategoryStats()
```

### 1.5.4 Паттерны и принципы разработки

– **Разделение ответственности (SoC)**: каждый JS-файл отвечает за строго определённую область — данные, утилиты, логика, UI;

– **DRY (Don't Repeat Yourself)**: вспомогательные функции вынесены в utils.js и используются из нескольких модулей;

– **Событийно-ориентированное программирование**: вся реакция на действия пользователя реализована через addEventListener;

– **Иммутабельность при фильтрации**: при фильтрации создаётся новый массив через filter(), исходный массив в localStorage не изменяется.

# 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

## 2.1 Создание проекта и структуры файлов

Первым этапом разработки стала организация структуры файлов проекта. Был создан репозиторий на GitHub (https://github.com/Timq11/project) и локально инициализирован Git. Структура директорий:

```
project/
■■■ projec-folder/
    ■■■ index.html        # ■■■■■■■■ ■■■■■■■■■
    ■■■ form.html         # ■■■■■■ ■■■■■■■■■ ■■■■■
    ■■■ about.html        # ■ ■■■■■■■■
    ■■■ stats.html        # ■■■■■■■■■■■
    ■■■ package.json      # NPM-■■■■■■■■■■■■■
    ■■■ css/
    ■   ■■■ style.css     # ■■■■■■ ■■■■■■■■■■■
    ■■■ js/
    ■   ■■■ storage.js    # ■■■■■■■ ■ localStorage
    ■   ■■■ utils.js      # ■■■■■■■■
    ■   ■■■ main.js       # ■■■■■■■■■ ■■■■■■■
    ■   ■■■ form.js       # ■■■■■■■■■■ ■■■■■■
    ■   ■■■ stats.js      # ■■■■■■■■■■■
    ■■■ docs/
        ■■■ documentation.md
```

Каждый HTML-файл подключает необходимые JS-модули через теги <script> в конце body. Порядок подключения важен: сначала storage.js и utils.js (базовые зависимости), затем специфичный для страницы модуль.

Для работы с документацией был настроен package.json:

```
{ "name": "goaltracker-docs", "version": "1.0.0", "devDependencies": {
"markdown-pdf": "^11.0.0" }, "scripts": { "docs": "markdown-pdf
docs/documentation.md" } }
```

## 2.2 Реализация слоя хранения данных (storage.js)

Модуль storage.js является уровнем абстракции над localStorage. Он инкапсулирует логику сериализации/десериализации данных и инициализации тестовыми данными.

### Функция getGoals()

Читает данные из localStorage по ключу "goals", десериализует JSON и возвращает массив объектов Goal. При отсутствии данных возвращает пустой массив.

```
function getGoals() { const data = localStorage.getItem('goals'); return data ?
JSON.parse(data) : []; }
```

### Функция saveGoals(goals)

Принимает массив целей, сериализует его в JSON и записывает в localStorage. Вызывается после любой операции изменения данных.

```
function saveGoals(goals) { localStorage.setItem('goals', JSON.stringify(goals)); }
```

### Функция initializeGoals()

Проверяет, есть ли данные в localStorage. Если нет — записывает три тестовых цели для демонстрации функциональности. Вызывается при событии DOMContentLoaded на главной странице.

Тестовые данные включают цели из категорий Study, Health и Finance с различным прогрессом (60%, 30%, 50%), что позволяет сразу продемонстрировать все возможности фильтрации и статистики.

## 2.3 Реализация вспомогательных функций (utils.js)

Модуль utils.js содержит утилитарные функции, используемые в нескольких модулях приложения. Их вынесение в отдельный файл соответствует принципу DRY.

### generateId()

Генерирует уникальный строковый идентификатор на основе временной метки и случайного числа. Вероятность коллизии практически исключена.

```
function generateId() { return Date.now().toString() +
Math.random().toString(36).substr(2, 9); }
```

### formatDate(dateString)

Преобразует дату из формата YYYY-MM-DD в формат локали ru-RU (ДД.ММ.ГГГГ). Используется при отображении дедлайна в карточках целей.

```
function formatDate(dateString) { const date = new Date(dateString); return
date.toLocaleDateString('ru-RU'); }
```

### calculateCompletedGoals(goals)

Принимает массив целей и возвращает количество выполненных (completed === true). Используется в модуле статистики.

```
function calculateCompletedGoals(goals) { return goals.filter(g =>
g.completed).length; }
```

### getCategoryStats(goals) и getMostPopularCategory(stats)

getCategoryStats() группирует цели по категориям и возвращает объект вида { Study: 3, Sport: 1, ... }. getMostPopularCategory() определяет категорию с наибольшим количеством целей путём итерации по этому объекту.

## 2.4 Реализация основной логики приложения (main.js)

Модуль main.js — самый объёмный и отвечает за отображение целей и все операции с ними на главной странице.

**renderGoals()**

Получает массив целей из localStorage и отображает их в DOM-элементе goalsContainer. Если массив пуст, выводит сообщение «Цели не найдены». Вызывается при загрузке страницы и после каждого изменения.

**createGoalCard(goal)**

Создаёт DOM-элемент карточки цели с помощью innerHTML. Карточка содержит:

– заголовок (title) и цветную метку категории;

– иконку приоритета (□ Высокий / □ Средний / □ Низкий);

– дедлайн в локализованном формате;

– прогресс-бар на основе CSS-переменной width;

– кнопки «Выполнена» / «Активировать» и «Удалить».

**deleteGoal(goalId)**

Запрашивает подтверждение через window.confirm(). При согласии фильтрует массив, удаляя цель с указанным id, и сохраняет результат. Затем вызывает renderGoals() для обновления UI.

**toggleGoalStatus(goalId)**

Находит цель по id через Array.find(), инвертирует поле completed (true → false или false → true), сохраняет и перерисовывает список.

**filterGoals()**

Читает значения из трёх элементов управления (#search, #categoryFilter, #statusFilter) и применяет последовательную цепочку фильтров к массиву целей. Метод применяет три независимых условия через Array.filter() без мутации исходных данных в localStorage.

## 2.5 Реализация формы создания цели (form.js)

Модуль form.js управляет формой создания цели на странице form.html. Он обрабатывает отправку формы, валидирует введённые данные и сохраняет новый объект Goal в localStorage.

### Процесс создания цели

– Пользователь заполняет форму и нажимает кнопку «Сохранить».

– Обработчик события submit вызывает event.preventDefault() для остановки стандартного поведения.

– Функция validateForm() проверяет все обязательные поля.

– При успешной валидации createGoalObject() формирует объект Goal.

– Объект добавляется в массив и сохраняется через saveGoals().

– Пользователю показывается сообщение об успехе.

– Через 1 секунду происходит редирект на index.html.

### Валидация данных

Функция validateForm() проверяет каждое поле формы. Поле прогресса дополнительно проверяется на диапазон 0–100. При ошибке функция возвращает false и выводит alert с описанием ошибки.

```
function validateForm() { const title =
document.getElementById('title').value.trim(); const category =
document.getElementById('category').value; const deadline =
document.getElementById('deadline').value; const priority =
document.getElementById('priority').value; const progress =
parseInt(document.getElementById('progress').value); if (!title) { alert('[redacted]'); return false; } if (!category) { alert('[redacted]'); return false; } if (!deadline) { alert('[redacted]'); return false; } if (!priority) { alert('[redacted]'); return false; }
if (isNaN(progress)||progress<0||progress>100) { alert('[redacted]: 0-100'); return
false; } return true; }
```

## 2.6 Реализация модуля статистики (stats.js)

Модуль stats.js вычисляет и отображает агрегированные показатели по всем целям. Он загружается только на странице stats.html.

### Вычисляемые метрики

– totalGoals — общее количество целей;

– completedGoals — количество выполненных (через calculateCompletedGoals());

– activeGoals — разница totalGoals − completedGoals;

– completionPercentage — доля выполненных в %, защищена от деления на ноль;

– mostPopularCategory — наиболее популярная категория (через getMostPopularCategory());

– categoryStats — разбивка количества целей по всем категориям.

Защита от деления на ноль реализована следующим образом:

```
const pct = totalGoals > 0 ? Math.round((completedGoals / totalGoals) * 100) : 0;
```

Функция renderStats() формирует HTML-строку с карточками метрик и вставляет её в элемент #statsContainer через innerHTML. Карточки категорий отображаются в виде сетки аналогично карточкам целей.

## 2.7 Разработка стилей приложения (style.css)

Файл style.css (4 218 байт) содержит все стили приложения. Для типографики используется шрифт Inter, подключённый из Google Fonts.

### Основные CSS-компоненты

– **Глобальные стили**: сброс отступов, box-sizing: border-box, цвет фона #f4f7fb, шрифт Inter, межстрочный интервал 1.6;

– **Навигация**: фон #1f3c88, flex-контейнер с gap и wrap, hover-эффекты с transition: 0.3s;

– **Сетка карточек**: CSS Grid с auto-fill, minmax(300px, 1fr), gap 1.5rem;

– **Карточка цели**: белый фон, border-radius 12px, box-shadow, hover: translateY(-3px) с увеличенной тенью;

– **Прогресс-бар**: полоса прогресса в % от ширины карточки, цвет #3b82f6;

– **Адаптивность**: @media (max-width: 768px) перестраивает навигацию, фильтры и сетку.

## 2.8 Обработка ошибок и валидация данных

В приложении реализованы следующие механизмы защиты от ошибок:

**Валидация на стороне клиента**

– Атрибут required на уровне HTML предотвращает отправку формы с пустыми полями;

– Атрибуты min="0" max="100" для поля прогресса ограничивают диапазон на уровне браузера;

– Функция validateForm() в JavaScript повторно проверяет все поля и выводит информативные сообщения;

– Проверка на NaN при parseInt для числовых полей.

**Защита от некорректного состояния**

– getGoals() возвращает пустой массив при отсутствии данных в localStorage, исключая ошибку «Cannot read properties of null»;

– Защита от деления на ноль в расчёте процента выполнения;

– Подтверждение перед удалением (window.confirm) исключает случайное удаление;

– initializeGoals() проверяет наличие данных перед инициализацией, не перезаписывая существующие цели пользователя.

**Пустые состояния (Empty States)**

При отсутствии целей, соответствующих фильтрам, в контейнере отображается информационное сообщение «Цели не найдены». Это предотвращает отображение пустого блока и улучшает UX.

# ЗАКЛЮЧЕНИЕ

В ходе учебной практики по профессиональному модулю ПМ2 «Прикладное программирование» было разработано веб-приложение GoalTracker — система постановки и отслеживания личных целей.

**Реализованный функционал**

– Создание целей с атрибутами: название, категория, дедлайн, приоритет, прогресс, описание;

– Отображение целей в виде адаптивной сетки карточек;

– Поиск по названию в режиме реального времени;

– Фильтрация по категории и статусу выполнения;

– Переключение статуса «активна / выполнена» одним кликом;

– Удаление цели с подтверждением;

– Модуль статистики с ключевыми метриками и разбивкой по категориям;

– Сохранение данных между сессиями браузера через localStorage;

– Адаптивный интерфейс для мобильных и десктопных устройств.

**Приобретённые навыки и компетенции**

– **HTML5**: семантическая разметка, формы с атрибутами валидации, data-атрибуты;

– **CSS3**: Grid Layout, Flexbox, адаптивный дизайн, анимации, CSS-переменные;

– **JavaScript ES6+**: модульная организация кода, работа с массивами, DOM API, события;

– **Web Storage API**: работа с localStorage, сериализация/десериализация JSON;

– **UI/UX**: проектирование интерфейса, цветовая схема, типографика;

– **Git/GitHub**: система контроля версий, работа с удалённым репозиторием;

– **Архитектура**: принципы SoC, DRY, событийно-ориентированного программирования.

**Направления для дальнейшего развития**

– Добавление серверной части (Node.js + Express) для хранения данных в облаке;

– Реализация аутентификации пользователей;

– Экспорт целей в форматы PDF и Excel;

– Push-уведомления о приближении дедлайнов;

– Тёмная тема оформления.

Практика достигла всех поставленных целей: разработано функциональное приложение, освоены ключевые веб-технологии, сформированы навыки самостоятельной разработки программного продукта. Приобретённые компетенции будут применяться в дальнейшей профессиональной деятельности.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MDN Web Docs. HTML: HyperText Markup Language [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/HTML – Дата обращения: 10.04.2026.

2. MDN Web Docs. CSS: Cascading Style Sheets [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/CSS – Дата обращения: 10.04.2026.

3. MDN Web Docs. JavaScript [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/JavaScript – Дата обращения: 10.04.2026.

4. MDN Web Docs. Web Storage API [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API – Дата обращения: 10.04.2026.

5. W3Schools. JavaScript Tutorial [Электронный ресурс]. – Режим доступа: https://www.w3schools.com/js/ – Дата обращения: 08.04.2026.

6. Флэнаган Д. JavaScript: Подробное руководство. – СПб.: Питер, 2021. – 1100 с.

7. Хавербеке М. Выразительный JavaScript. – 3-е изд. – М.: Диалектика, 2022. – 480 с.

8. CSS-Tricks. A Complete Guide to CSS Grid [Электронный ресурс]. – Режим доступа: https://css-tricks.com/snippets/css/complete-guide-grid/ – Дата обращения: 07.04.2026.

9. CSS-Tricks. A Complete Guide to Flexbox [Электронный ресурс]. – Режим доступа: https://css-tricks.com/snippets/css/a-guide-to-flexbox/ – Дата обращения: 07.04.2026.

10. Google Fonts. Inter Font Family [Электронный ресурс]. – Режим доступа: https://fonts.google.com/specimen/Inter – Дата обращения: 05.04.2026.

11. GitHub Docs. About Git [Электронный ресурс]. – Режим доступа: https://docs.github.com/ru/get-started/using-git/about-git – Дата обращения: 10.04.2026.

12. npm Documentation. package.json [Электронный ресурс]. – Режим доступа: https://docs.npmjs.com/cli/v10/configuring-npm/package-json – Дата обращения: 09.04.2026.

# ПРИЛОЖЕНИЯ

## Приложение А – Листинг файла storage.js

```
const STORAGE_KEY = 'goals'; function getGoals() { const data =
localStorage.getItem(STORAGE_KEY); return data ? JSON.parse(data) : []; } function
saveGoals(goals) { localStorage.setItem(STORAGE_KEY, JSON.stringify(goals)); }
function initializeGoals() { if (localStorage.getItem(STORAGE_KEY)) return; const
defaultGoals = [ { id: generateId(), title: '[redacted] [redacted] [redacted]
JavaScript', category: 'Study', deadline: '2026-05-01', priority: 'High', progress:
60, description: '[redacted] [redacted] GoalTracker', completed: false, createdAt:
'2026-01-15' }, { id: generateId(), title: '[redacted] [redacted] 3 [redacted] [redacted] [redacted]',
category: 'Health', deadline: '2026-04-30', priority: 'Medium', progress: 30,
description: '', completed: false, createdAt: '2026-01-20' }, { id: generateId(),
title: '[redacted] [redacted] [redacted] [redacted]', category: 'Finance', deadline:
'2026-07-01', priority: 'Low', progress: 50, description: '[redacted] 80 000 [redacted]',
completed: false, createdAt: '2026-02-01' }, ]; saveGoals(defaultGoals); }
```

## Приложение Б – Листинг файла utils.js

```
function generateId() { return Date.now().toString() +
Math.random().toString(36).substr(2, 9); } function formatDate(dateString) { const
date = new Date(dateString); return date.toLocaleDateString('ru-RU'); } function
calculateCompletedGoals(goals) { return goals.filter(g => g.completed).length; }
function getCategoryStats(goals) { return goals.reduce((acc, goal) => {
acc[goal.category] = (acc[goal.category] || 0) + 1; return acc; }, {}); } function
getMostPopularCategory(stats) { return Object.entries(stats).reduce( (max, [cat,
cnt]) => cnt > max[1] ? [cat, cnt] : max, ['', 0] )[0]; }
```

## Приложение В – Структура объекта Goal (JSON-пример)

```
{ "id": "1712345678abcdef", "title": "[redacted] [redacted] [redacted] JavaScript",
"category": "Study", "deadline": "2026-05-01", "priority": "High", "progress": 60,
"description": "[redacted] [redacted] GoalTracker", "completed": false, "createdAt":
"2026-01-15" }
```